

EE-2003

Computer Organization and Assembly Language

Week 09 – Lecture 18

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Subroutines & Why We Need?
- 3 CALL and RET — Basic Concept
- 4 Our First Subroutine – Concept

Quick Review: What was in the Last Lecture?

- Bitwise Logical Operations
- Masking Operations

Why We Need Subroutines

Background:

- We have basic assembly tools: conditional jumps, loops, and arithmetic.
- Using these, we built useful algorithms:
 - Bubble Sort
 - Multiplication algorithm
- The multiplication routine is more flexible than the built-in MUL instruction because it can be extended to 32- or 64-bit.

Problem:

- Programs often need sorting or multiplication many times.
- Copying the same code at 100 places is infeasible and unmaintainable.

Code Reuse Challenge

- One idea: place the routine at one label and JMP there whenever needed.
- Problem → after completion, the processor does not know where to return.
- JMP has a single fixed destination — not suitable when many callers need to come back to different points.

Execution Analogy:

- Program = a continuous thread of instructions.
- JMP = permanent cut in that thread (a one-way road).
- We need a **temporary diversion** that goes out and returns — like a roundabout.

Temporary Diversion — The Idea

- We want to enter a block of code, perform a task, then return to where we left.
- The diversion must be **temporary**, not permanent.
- Each algorithm (e.g., Bubble Sort, Multiply) can exist once in memory.
- We simply **divert** → **execute** → **return**.

Assembly Solution:

- Permanent diversion → JMP
- Temporary diversion → CALL ... RET

CALL and RET — Basic Concept

CALL instruction:

- Temporarily diverts execution to a labeled routine.
- Saves the address of the next instruction (after CALL) on the stack.

RET instruction:

- Retrieves the saved address from the stack.
- Returns control to the point immediately after CALL.

Effect:

- Enables one-time code definition, reusable many times.
- Forms the foundation of modular programming in assembly.

CALL vs JMP — Comparison

Feature	CALL	JMP
Diversion type	Temporary	Permanent
Return path	Automatic via RET	None
Stack usage	Stores return address	None
Typical use	Subroutines / Functions	Branching / Loops

CALL + RET act as a controlled round-trip, whereas JMP is one-way.

Independence of CALL and RET

- Technically, each instruction can exist alone.
- CALL does not require a matching RET — though the program may never return.
- RET can appear without CALL — it simply pops an address from the stack.
- In normal structured programs, they are used as a logical pair.
- Advanced techniques sometimes exploit them individually for control tricks.

Parameters — Making Subroutines Flexible

- A fixed routine is limited: it always processes the same data.
- Real programs require different data → need **parameters**.
- Example: sort various arrays, ascending or descending, signed or unsigned.
- Parameters are the information passed from the **caller** to the **subroutine**.

Parameter Passing Methods:

- Through CPU **registers** (AX, BX, CX ...)
- Through shared **memory** variables
- Through the **stack** (for more complex calls)

Subroutine Call Process — Visual Summary

- ① Main program executes normally.
- ② `CALL label` → address of next instruction pushed to stack.
- ③ Control transfers to the subroutine at `label`.
- ④ Subroutine performs its task.
- ⑤ `RET` → pops saved address and resumes the main flow.

Section 5.1 – Summary

- **Problem:** repeated code for common tasks (sort, multiply) is inefficient.
- **Solution:** subroutines using CALL and RET.
- JMP = permanent diversion; CALL + RET = temporary diversion.
- **Parameters** make subroutines reusable for multiple datasets.
- The stack enables the automatic return mechanism.

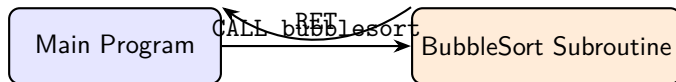
“Subroutines bring modularity, reusability, and clarity to assembly programming.”

5.2 Our First Subroutine – Concept

- We now convert our Bubble Sort code into a **subroutine**.
- A label marks the start of the routine, and a RET marks its end.
- The main program will CALL this subroutine whenever sorting is needed.
- After completing, control **returns automatically** to the next instruction.

Key Idea

The code can be reused multiple times without repetition.



Example 5.1 – Bubble Sort as a Subroutine (Part 1)

```
[org 0x0100]

        jmp start

data:    dw    60, 55, 45, 50, 40, 35, 25, 30, 10, 0
swap:    db    0

bubblesort:  dec    cx                ; last element not compared
            shl    cx, 1              ; multiply by 2 (word array)

mainloop:   mov     si, 0              ; initialize index
            mov     byte [swap], 0    ; reset swap flag

innerloop:  mov     ax, [bx+si]        ; load element
            cmp     ax, [bx+si+2]     ; compare with next
            jbe     noswap            ; skip if in order

            mov     dx, [bx+si+2]     ; swap elements
            mov     [bx+si], dx
            mov     [bx+si+2], ax
            mov     byte [swap], 1
```

Example 5.1 – Bubble Sort as a Subroutine (Part 2)

```
noswap:      add    si, 2                ; move to next pair
             cmp    si, cx
             jne    innerloop

             cmp    byte [swap], 1      ; check if swapped
             je     mainloop

             ret                        ; return to caller

start:       mov    bx, data             ; base address
             mov    cx, 10              ; number of elements
             call   bubblesort          ; call subroutine

             mov    ax, 0x4c00
             int    0x21
```

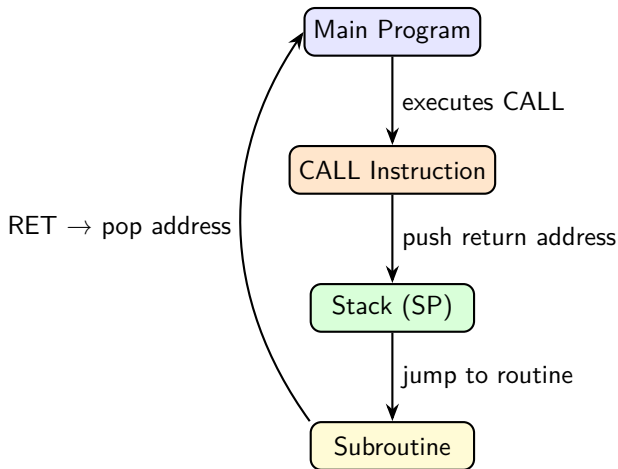
Understanding the Routine

- BX holds the base address of the array.
- CX holds the number of elements.
- The routine decrements CX because the last element needs no comparison.
- Then multiplies CX by two (since it's a word array).
- Inside the debugger:
 - BX = 0103h → address of data.
 - CX = 000Ah → 10 elements.
 - Step over CALL → sorting completes instantly.
 - Step into CALL → executes each instruction of bubblesort.

How CALL and RET Use the Stack

- CALL automatically:
 - Pushes the return address onto the stack.
 - Changes IP to the subroutine label.
 - Decrements SP by 2.
- RET:
 - Pops the stored address from the stack.
 - Restores SP to its previous value.
 - Resumes execution after the CALL.
- Example: After CALL, SP changes from FFFE $\rightarrow$ FFFC, and 0150 (return address) is placed on the stack. When RET executes, 0150 is popped back into IP $\rightarrow$ resumes execution.

Stack Operation During CALL and RET



Stack automatically coordinates between CALL and RET.

Section 5.2 – Summary

- Bubble sort is converted into a **subroutine**.
- CALL and RET enable reusable, modular code.
- Parameters are passed in registers (BX, CX).
- Stack stores the return address automatically.
- Execution flow:

Main Program → CALL → Subroutine → RET → Main Program

“CALL and RET bring structured programming to assembly.”

Thank You!

Any Questions?

- **Topics Covered:** Subroutines, CALL and RET, Parameter Passing, and Stack Operations.
- **Key Takeaways:**
 - **Subroutines:** Enable modular and reusable code in assembly.
 - **CALL & RET:** Work together for temporary diversion and automatic return.
 - **Stack:** Used to store the return address — ensuring safe execution flow.
 - **Parameters:** Passed through registers or memory to make routines flexible.
 - **Practical Relevance:** Foundation for structured programming, function calls, and interrupt handling in higher-level architectures.

“Subroutines bring structure, reusability, and precision to low-level programming.”